

Feature-Based Lightweight ML Reranker 设计说明

CPS3830 Machine Learning Final Project - RAG_Medical_Reranker

用途：这份文档用于解释本项目中 Lightweight ML reranker 的 feature-based 设计。它是后续写 report、交接给队友、或发给 ChatGPT 继续分析时的自包含说明。文档同时说明 Department Match feature 与旧 RAG 中 Department Bonus heuristic 的关系。

0. 项目定位与相关文件路径

本项目关注医疗 RAG pipeline 中的 reranking 阶段。统一输入是一个 candidate-level benchmark：每个 question 对应 10 个 candidate contexts，其中 1 个 positive 和 9 个 negatives。Lightweight ML reranker 的任务不是生成医学回答，而是为每个 question-context pair 预测相关性分数，并在每个 qid 内部重新排序。

类别	路径 / 文件名	作用
统一候选集	data/processed/candidates.csv	所有 baseline、ML reranker、BCE/BGE reranker 的共享输入。当前规模为 300 qids / 3000 rows，每个 qid 有 10 candidates。
特征工程	src/ml/feature_engineering.py	把原始 question 和 candidate_context 以及 metadata 转换成 6 个数值特征。
Lightweight ML 训练	src/ml/train_lightweight_ml.py	按 qid 切分 train/test，训练 Logistic Regression 或 Random Forest，并输出 ml_scores.csv。
Embedding baseline	src/embedding_baseline.py	直接使用 candidates.csv 中的 embedding_score 排序。
统一评估	src/evaluate.py	读取不同 method 的 score file，计算 P@K、MRR、NDCG@5 和 latency。
工具函数	src/utils.py	包含 validate_candidates、rank_scores、ensure_parent 等公共函数。
ML 输出	results/ml_scores.csv	Lightweight ML reranker 对每个 candidate 的 score 和 rank。
ML 分类指标	results/ml_classification_metrics.csv	Logistic Regression 在 qid-level held-out test set 上的 classification metrics。
总结果	results/summary_metrics.csv	embedding_baseline、lightweight_ml、bce 等方法的统一 ranking metrics。

当前已完成的主结果：embedding_baseline 的 P@1 = 0.9967，lightweight_ml 的 P@1 = 1.0000，BCE 的 P@1 = 1.0000。由于 embedding baseline 已经接近满分，当前 benchmark 主要证明 pipeline 可运行；后续若要体现 reranker 优势，需要新增更难的 hard-negative benchmark。

1. Feature-Based Reranker 是什么

本项目中的 Lightweight ML reranker 是 feature-based supervised model。它不直接让 LLM 阅读全文，也不 fine-tune transformer。它的核心思想是：先用 Python 代码把每一条 question-context pair 转换成一组可解释的数值特征，然后用传统监督学习模型学习这些特征和 label 之间的关系。

```
raw question + candidate_context + metadata
-> Python feature engineering
-> numerical feature vector
-> Logistic Regression / Random Forest
```

```
-> relevance probability as score
-> rank candidates within each qid
```

这里的监督信号来自 candidates.csv 中的 label: label = 1 表示该 candidate 是原始匹配的 positive context; label = 0 表示 hard/random negative。模型不是学习完整排序列表, 而是学习一个 pointwise binary classification task: 给定一个 question-context pair, 预测它是否 relevant。预测出的 positive probability 被用作 reranking score。

2. 当前使用的 6 个特征

特征工程文件是 src/ml/feature_engineering.py。当前 FEATURE_COLUMNS 包含 6 个特征:

```
FEATURE_COLUMNS = [
    "embedding_score",
    "keyword_overlap",
    "department_match",
    "question_length",
    "context_length",
    "length_ratio",
]
```

Feature	来源	是否使用 LLM	含义与作用
embedding_score	candidates.csv 已有字段	不使用 LLM generation; 使用 embedding model	question embedding 与 candidate_context embedding 的 cosine similarity, 是最强的语义相似度信号。
keyword_overlap	feature_engineering.py 中 Python 规则计算	否	将 question 和 context 拆成中文字符 bigram, 计算 Jaccard overlap, 表示字面关键词重合程度。
department_match	question_department 与 department 比较	否	如果问题科室和候选科室一致, 则为 1, 否则为 0。表示医学科室层面的匹配。
question_length	len(question)	否	question 字符长度, clip 到 500 后归一化。
context_length	len(candidate_context)	否	candidate_context 字符长度, clip 到 1500 后归一化。
length_ratio	question/context 长度比例	否	$\min(\text{lengths}) / \max(\text{lengths})$, 表示两段文本长度是否差异过大。

重要说明: Lightweight ML 的 feature extraction 本身主要是 Python 规则与统计计算, 不调用 LLM 做生成, 也不让 LLM 判断相关性。唯一与模型语义表征相关的是 embedding_score, 它来自 embedding model, 而不是聊天式大语言模型的回答生成。

3. embedding_score: 语义相似度特征

embedding_score 是当前 6 个 features 中最重要的语义特征。它并不是 Logistic Regression 自己计算的, 而是在 candidate generation / embedding baseline 阶段已经写入 candidates.csv。其含义是 question 与 candidate_context 的 embedding cosine similarity。

```
question -> Ollama nomic-embed-text -> question vector
candidate_context -> Ollama nomic-embed-text -> context vector
cosine(question vector, context vector) -> embedding_score
```

这个特征使用的是 embedding model, 不是 LLM generation。它把文本转换成向量, 然后通过 cosine similarity 衡量语义接近程度。对于 RAG reranking, 这个特征天然很强, 因为 RAG 的初始检索本来就是基于向量相似度。

在本项目当前 controlled candidate benchmark 中，embedding_score 已经几乎可以单独解决排序问题：embedding_baseline 的 Precision@1 = 0.9967。这说明当前候选集偏容易，也解释了为什么 Logistic Regression 和 BCE 的提升空间很小。

4. keyword_overlap：字面关键词重合特征

keyword_overlap 是纯 Python 规则特征，用于补充 embedding_score。它不理解深层医学语义，而是衡量 question 与 candidate_context 在字符层面的重合程度。

```
1. clean_text(text)
2. extract Chinese characters / letters / digits
3. build character bigrams
4. keyword_overlap = len(q_bigrams & c_bigrams) / len(q_bigrams | c_bigrams)
```

例如，question 中含有“胃癌”“治疗”“化疗”等字面片段，而 candidate_context 中也出现类似片段，则 keyword_overlap 会更高。反之，如果 candidate_context 是“小儿发烧”“中耳炎”等跨科室内容，则 overlap 会低。

这个特征的价值在于：embedding_score 负责语义接近，keyword_overlap 负责表面关键词接近。两者结合后，Logistic Regression 可以学习到“语义相似 + 关键词匹配”通常更可能是 positive。

5. department_match：科室匹配特征

department_match 是当前 feature-based reranker 中与医学领域最直接相关的 metadata feature。它的计算方式是：

```
department_match = 1.0 if question_department == department else 0.0
```

其中 question_department 来自 candidates.csv，表示该 question 的标准科室；department 表示 candidate_context 所属科室。如果二者相同，则 department_match = 1，否则为 0。

这个特征背后的医学直觉是：同科室 candidate 通常比跨科室 candidate 更可能相关。例如，胃癌问题更可能匹配 oncology context，而不是 pediatric context。

但这个特征也带来一个重要实验解释：当前 candidates.csv 包含 5 个跨科室 random negatives，所以 department_match 能够很容易帮助模型排除部分 random negatives。这也是当前 benchmark 偏容易的原因之一。

6. 长度类特征：question_length、context_length、length_ratio

长度类特征不是医学语义特征，而是辅助性的统计特征。它们帮助模型捕捉文本长度模式，避免某些异常短或异常长的 candidate 对排序产生影响。

Feature	计算方式	解释
question_length	$\min(\text{len}(\text{question}), 500) / 500$	将问题长度压缩到 0 到 1 附近，避免长问题数值过大。
context_length	$\min(\text{len}(\text{candidate_context}), 1500) / 1500$	将候选 context 长度归一化。
length_ratio	$\min(q_len, c_len) / \max(q_len, c_len)$	衡量 question 和 context 长度是否过于不匹配。

这些特征单独看不会决定相关性，但在 Logistic Regression 中可以作为弱信号参与校准。例如，如果某些 positive context 通常具有较完整的标题、患者问题和医生回答，context_length 可能间接提供信息。

7. 禁止使用的字段与数据泄漏控制

Lightweight ML 的一个重要设计原则是避免 data leakage。以下字段不能作为 ML features：

字段	为什么不能作为 feature
label	这是监督学习的目标 y。如果作为 feature，模型等于直接看到答案。
source_type	它暴露 candidate 的采样来源，例如 positive、hard_negative、random_negative。使用它会让模型学会数据构造规则，而不是相关性。
qid	问题 ID 只是样本标识，不具备可泛化意义。
context_id	候选 ID 只是样本标识，不应作为相关性信号。

因此，训练脚本 src/ml/train_lightweight_ml.py 中，X 由 build_features(df) 生成，y 由 label 生成。label 只作为监督目标，不进入 FEATURE_COLUMNS。

8. Logistic Regression 的训练逻辑

Lightweight ML 将 reranking 问题转化为 pointwise binary classification。每一行 question-context pair 是一个训练样本，label = 1 表示 relevant，label = 0 表示 non-relevant。

src/ml/train_lightweight_ml.py 中支持 Logistic Regression 和 Random Forest。当前正式报告结果使用 Logistic Regression，主要原因是：模型简单、可解释、适合低维 tabular features，并且 inference latency 极低。

```
candidates.csv
-> split qids into train/test
-> build_features(train) -> X_train
-> train["label"] -> y_train
-> LogisticRegression(class_weight="balanced")
-> predict_proba(X_score)[: , 1]
-> rank_scores within each qid
-> results/ml_scores.csv
```

注意，数据切分是按 qid 进行，而不是按 row 随机切分。这样可以避免同一个 question 的部分 candidates 出现在 train、另一部分 candidates 出现在 test，从而造成评估泄漏。当前默认 test_size = 0.25，因此 300 个 qids 中约 225 个用于训练，75 个用于测试 classification metrics。

```
PYTHONPATH=. python src/ml/train_lightweight_ml.py \
--model logreg \
--score-all \
--out results/ml_scores.csv \
--metrics-out results/ml_classification_metrics.csv
```

9. --score-all 的含义

--score-all 是为了让输出文件与其他方法保持同一 evaluation scope。默认情况下，ML 训练脚本只会对 test qids 生成 scores；加入 --score-all 后，模型仍然基于 train qids 训练，但会对全部 candidates.csv rows 生成 scores。

设置	输出范围	用途
不加 --score-all	只输出 held-out test qids	更严格的 ML test-only evaluation。
加 --score-all	输出全部 300 qids / 3000 rows	与 embedding_baseline、BCE 在同一 full benchmark 上比较。

当前项目正式结果使用 --score-all，因此 results/ml_scores.csv 有 3000 rows。报告中应说明：classification metrics 来自 qid-level held-out test set；full ml_scores.csv 用于统一 ranking comparison。

10. Department Match Feature 与 Department Bonus Heuristic 的关系

Department Match 和 Department Bonus 都利用“科室一致性”这个医学先验，但二者不是同一件事。

项目	Department Match Feature	Department Bonus Heuristic
所在方法	Lightweight ML reranker	Rule-based embedding baseline ablation
形式	一个输入特征，值为 0 或 1	直接修改排序分数，例如 <code>score = embedding_score + bonus</code>
是否学习权重	是。Logistic Regression 学习它的系数。	否。bonus 大小由人工设定。
是否 ML	是，作为 ML feature 参与训练。	不是，是启发式规则。
是否应写入 <code>embedding_score</code>	不应写入。作为单独 feature 使用。	也不应覆盖原 <code>embedding_score</code> ，应作为单独 method 输出。

当前 feature-based reranker 已经使用 `department_match` 作为一个学习特征。模型可以根据训练数据自动决定科室匹配应该有多重要。相比之下，Department Bonus 是旧 RAG 系统中的人工规则，例如如果 `question_department == department`，就给 `embedding_score` 加 0.05。

如果后续要把旧 RAG 的 department bonus 纳入实验，推荐做成单独 ablation method，而不是混进当前 `embedding_baseline` 或 ML features。

```
score = embedding_score + 0.05 if question_department == department else embedding_score
method = embedding_department_bonus
output = results/department_bonus_scores.csv
```

这样可以比较：`embedding_only`、`embedding + department_bonus`、`lightweight_ml`、`BCE/BGE`。它能回答一个实际问题：旧 RAG 里的科室规则到底能贡献多少？但必须注意，department bonus 不应覆盖 `candidates.csv` 中原始 `embedding_score`，否则会污染 baseline。

11. Lightweight ML 与 BCE 的区别

本项目中 Lightweight ML 和 BCE 的最大区别在于：Lightweight ML 先把文本压缩成少量手工特征，再用传统 ML 模型；BCE 则直接输入 question-context text pair，由 transformer reranker 输出 relevance score。

维度	Lightweight ML	BCE Reranker
输入	6 个 engineered numerical features	完整 question + candidate_context 文本
模型类型	Logistic Regression / Random Forest	Pretrained transformer reranker
是否训练	在本项目数据上训练 supervised classifier	不训练，直接做 inference scoring
是否直接理解全文	不直接理解，依赖特征	直接读取文本 pair
速度	极快，当前 <code>avg_latency_ms = 0.0002</code>	较慢，当前 <code>avg_latency_ms = 0.5874</code>
优势	可解释、低延迟、易复现	语义建模能力更强
局限	依赖人工特征，困难语义负例可能弱	需要 GPU/较高计算资源，延迟更高

12. 当前结果如何解释

Method	P@1	MRR	NDCG@5	Avg Latency ms/pair	解释
--------	-----	-----	--------	---------------------	----

embedding_baseline	0.9967	0.9983	0.9988	0.0000	已经接近满分，说明当前 candidates 较容易。
lightweight_ml	1.0000	1.0000	1.0000	0.0002	达到满分，且极低延迟。
bce	1.0000	1.0000	1.0000	0.5874	达到满分，但计算成本高于 ML。

这些结果说明 pipeline 成功，但也说明 benchmark 需要升级。不能在报告中过度声称 ML/BCE 显著提升，因为 baseline 已经接近满分。更准确的说法是：在 controlled setting 下，Lightweight ML 和 BCE 可以把已经很强的 embedding baseline 推到 perfect ranking；但为了更充分验证 reranker 价值，需要构造 retrieval-hard candidates。

13. Report 中推荐的书面表述

可以在 Methodology 中这样写：

The lightweight ML reranker follows a feature-based supervised learning approach. Instead of directly fine-tuning a large language model, each question-context pair is first converted into a small set of engineered numerical features, including embedding similarity, lexical overlap, department matching, and length-based statistics. A Logistic Regression classifier is trained to predict the probability that a candidate context is relevant to the query. This probability is then used as the reranking score within each qid.

可以在 Discussion 中这样写：

Although the lightweight model does not perform deep semantic reasoning over the full text, it provides an interpretable and extremely low-latency reranking baseline. Its strong performance in the current controlled benchmark is partly supported by the high quality of the embedding_score feature and the relatively easy negative sampling strategy. Therefore, the result should be interpreted as evidence that the reranking pipeline is functional and efficient, rather than as definitive proof of clinical-level generalization.

可以在 Department Bonus ablation 中这样写：

Department information can be used in two different ways. In the lightweight ML reranker, department_match is treated as a learnable feature whose importance is estimated from data. In contrast, a department bonus baseline would manually add a fixed score bonus to candidates from the same department. Keeping these two approaches separate allows us to compare learned reranking against rule-based retrieval heuristics without contaminating the original embedding baseline.

14. 后续传给 ChatGPT 时需要附带的信息

如果以后把这份 PDF 发给 ChatGPT 继续写 report 或设计 hard benchmark，建议同时提供以下文件内容或路径：

优先级	文件	为什么重要
最高	src/ml/feature_engineering.py	定义 feature-based reranker 的全部特征。
最高	src/ml/train_lightweight_ml.py	定义训练、qid split、score-all、输出逻辑。
最高	src/utils.py	定义 validate_candidates、rank_scores 等公共规范。
高	src/embedding_baseline.py	定义 embedding baseline 如何读取 embedding_score 并排序。
高	src/evaluate.py	定义 ranking metrics 和 score file 格式校验。
中	src/rerankers/bce_reranker.py	定义 BCE transformer reranker 的 text-pair inference。

中	data/processed/candidates.csv 的 head 和 value_counts	用于理解当前 benchmark 结构。
中	results/summary_metrics.csv	用于理解当前实验结果。

15. 一句话总结

本项目中的 Lightweight ML reranker 是一个传统 feature-based supervised reranker：它通过 Python 规则和预先计算的 embedding_score，把每个 question-context pair 转换为 6 个可解释数值特征，再用 Logistic Regression 学习这些特征与 relevance label 的关系。它不使用 LLM 生成答案，也不直接让大模型判断相关性；Department Match 是 ML 模型中的可学习特征，而 Department Bonus 应作为单独的 rule-based baseline ablation 来评估。